

01 · pi 命令入口、CLI 参数解析与模式分流

从 npm bin 到 Interactive / Print / JSON / RPC 的完整控制流

分析版本: 027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

`pi` 的安装入口是 `packages/coding-agent/dist/cli.js`，对应源码 `src/cli.ts`；完整 CLI 生命周期真正落在 `src/main.ts::main(args)`。参数解析由 `src/cli/args.ts::parseArgs` 完成，而最终运行模式由 `main.ts` 中的 `resolveAppMode` 判定。JSON 不是独立 Agent 执行器，而是 `runPrintMode` 的 JSONL 输出模式。

1. 入口不是一个点，而是三层

阅读 CLI 时最常见的误区是把 `package.json` 的 `bin`、`src/cli.ts` 和 `main()` 混成“入口”。它们分别承担可执行文件映射、进程级初始化和业务级生命周期编排。

```
npm bin: pi -> dist/cli.js
source:  src/cli.ts
          |
          v
        main(process.argv.slice(2))
          |
          v
      src/main.ts::main(args)
```

层	位置	职责	是否业务入口
安装/发布层	<code>packages/coding-agent/package.json -> bin.pi</code>	把 shell 命令 <code>pi</code> 映射到构建产物 <code>dist/cli.js</code>	否
进程层	<code>packages/coding-agent/src/cli.ts</code>	设置 <code>process.title</code> 、环境变量、HTTP dispatcher，调用 <code>main</code>	半入口
应用层	<code>packages/coding-agent/src/main.ts::main</code>	子命令、参数、资源、模型、Session、模式创建与分流	是

2. main() 之前为什么还有特殊子命令

`main()` 并不是一上来就 `parseArgs`。 `package/config/auth` 一类命令先被专用 handler 尝试消费。这样做把“管理型命令”和“启动 Agent 会话”分开：安装包、修改配置、认证等操作不需要创建模型、会话和 TUI。

```
main(args)
-> handlePackageCommand(args)
-> handleConfigCommand(args)
-> runCredentialPrintCommand(args)
-> parseArgs(args)
-> resolveAppMode(...)
-> create Session / Runtime
-> dispatch mode
```

3. parseArgs 的模型：手写顺序扫描器

parseArgs 不是 Commander/Yargs 风格的声明式 parser，而是一个 for 循环顺序扫描 argv。带值参数通过推进索引消费下一个 token；普通字符串进入 messages；以 @ 开头的参数进入 fileArgs。未知长 flag 会保留给扩展系统，未知短 flag 则生成 diagnostic。

输入示例	解析结果	关键含义
--model openai/gpt-x hello	model=openai/gpt-x; messages=[hello]	带值参数消费下一个 token
-p "explain"	print=true; messages=[explain]	-p 可以吸收紧随其后的普通消息
@src/main.ts review	fileArgs=[src/main.ts]; messages=[review]	文件引用与普通 prompt 分开收集
--my-ext-flag value	unknownFlags 保存	给 Extension CLI flag 二次解析留入口

4. resolveAppMode 的真实优先级

最终应用模式不是简单等于 --mode。内部 AppMode 是 interactive / print / json / rpc；CLI 的 Mode 参数是 text / json / rpc。两者故意不同。

```
if mode == rpc      -> rpc
else if mode == json -> json
else if print || !stdinTTY || !stdoutTTY -> print
else                -> interactive
```

因此优先级是 rpc > json > print > interactive。`--mode text` 在正常 TTY 中并不会强制单次输出；真正使其成为 Print 的是 -p 或非 TTY。

5. 四种模式的分流点与共享核心

模式	触发	最终调用	I/O 契约
Interactive	默认且 stdin/stdout 均为 TTY	new InteractiveMode(runtime).run()	终端 Editor/TUI + 流式事件
Print/Text	-p 或管道/重定向	runPrintMode(runtime, {mode:"text"})	最终 assistant 文本
JSON	--mode json	runPrintMode(runtime, {mode:"json"})	Session/Agent 事件逐行 JSONL
RPC	--mode rpc	runRpcMode(runtime)	stdin JSON command / stdout JSON response+event

四种模式共享同一个 AgentSessionRuntime/AgentSession。模式层只是不同的 I/O 适配器，这一点决定了后续阅读时应把业务核心放在 AgentSession，而不是分别追四套 Agent 实现。

6. 非 TTY 为什么自动退化到 Print

交互 TUI 假设 stdin 和 stdout 都是可控制的终端。当 stdin 被 pipe 占用时，输入不再是键盘事件；当 stdout 重定向到文件时，ANSI 增量渲染也失去意义。resolveAppMode 因而把任一非 TTY 情况视作非交互执行。RPC 是例外，因为它有意把 stdin/stdout 当协议通道，所以 main 不会对 RPC 调 readPipedStdin。

7. JSON 模式的一个重要设计细节

JSON 复用 runPrintMode，说明“执行一次 prompt 并退出”和“如何编码输出”被拆成两个维度。Text 只在最后打印 assistant 文本；JSON 在 session.subscribe 上输出 header 与事件。这个设计避免了复制 Agent 生命周期。

8. 边界条件与调试关注点

- 非法 --mode 在 parseArgs 阶段被消费但不一定立即报错；阅读错误处理时要继续看 diagnostics/调用方。
- RPC 不接受 @file 这种一次性 CLI 语义；文件/内容应通过 RPC 协议交互。
- 普通 `pi "hello"` 在 TTY 中仍是 Interactive，不是 Print。
- stdin pipe 与 stdout redirect 都会改变 AppMode；复现模式问题时必须记录 isTTY。
- 扩展 CLI flags 的存在解释了 unknown long flags 为什么不是一律报错。

动态验证方案

实验 1：确认真实入口

假设：dist/cli.js 与 src/cli.ts 对应，业务入口为 main()

操作：在 src/cli.ts、main.ts、args.ts 的入口分别打断点，执行 `pi --mode json hello`。

预期：命中 cli.ts -> main -> parseArgs -> resolveAppMode；最终进入 runPrintMode(json)。

若不一致：若 dist sourcemap 或构建版本不一致，先核对安装包版本和 commit。

实验 2：验证 TTY 分流

假设：非 TTY 会自动进入 Print

操作：分别运行 `pi "x"`、`echo x | pi`、`pi "x" > out.txt` 并记录 appMode。

预期：前者 interactive，后二者 print。

若不一致：若不一致，检查 shell/IDE 是否给进程分配伪 TTY。

源码证据索引

文件	关键符号/用途	本报告结论
packages/coding-agent/package.json	bin.pi / build	证明命令到 dist/cli.js 的发布映射
packages/coding-agent/src/cli.ts	main(process.argv.slice(2))	进程层入口

packages/coding-agent/src/main.ts	main / resolveAppMode / runPrintMode / runRpcMode	应用生命周期与模式分流
packages/coding-agent/src/cli/args.ts	parseArgs / Args / Mode	参数扫描与 messages/fileArgs/unknownFlags
packages/coding-agent/src/modes/rpc/rpc-mode.ts	runRpcMode	RPC stdin/stdout 协议入口

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/coding-agent/src/cli.ts	确认最薄进程入口	是
2	packages/coding-agent/src/main.ts	建立完整 CLI 生命周期	是
3	packages/coding-agent/src/cli/args.ts	理解参数语义及扩展 flag	是
4	packages/coding-agent/src/modes/print-mode.ts	对比 text/json 输出	是
5	packages/coding-agent/src/modes/rpc/rpc-mode.ts	理解 RPC 为何独占 stdin/stdout	否